ESE 5370: Final Project Report

Parth Sarkar

Wednesday, April 29th, 2026

1 Introduction

Memory safety is a key priority for programs written in C. The burden of programing defensively against malicious use of undefined behavior lies squarely on the programmer, leading to mistakes that can be exploited by user input to the program. While the problem is pervasive and fundamentally a consequence of C semantics, the Softbound+CETS project offers the strong, formally-verified guarantee of *full* memory safety. Softbound+CETS instruments C code to create and update metadata for each pointer in the program at runtime. Upon a dereference for a pointer, the pointer's metadata is cross-referenced to ensure that the dereference occurs "within bounds" and is not a use-after-free.

However, this approach has two limitations. First, from a performance perspective, it is highly expensive to branch and check any condition on every dereference; [INSERT (revisited)] cites a 140% overhead on the SPEC CPU 2017 C benchmark suite. However, more pressingly, Softbound+CETS does not offer binary-level compatibility. Hence, if any C code that has not been compiled with Softbound+CETS instrumentation is linked into a protected project, the entire executable is susceptible to memory privilege escalation.

With these problems in mind, my final project first attempts to concretize the problem of privilege escalation due to foreign-linked code with a working attack that permits a buffer-overflow write (one that would normally be stopped by Softbound+CETS instrumentation if compiled with protection). As a proposed solution, I implement and compare two encryption methods on the metadata table that Softbound+CETS uses to propagate metadata on pointers stored into memory: a simple XOR encryption, and a more complicated AES encryption method. On the performance front, I implement and formally prove important properties of a global dataflow analysis in LLVM 12 and the Rocq interactive theorem prover, respectively. I integrate the results of this analysis into the Softbound+CETS pass and demonstrate correctness and tangible performance improvements. Ultimately, I hope my project makes the case that dynamic enforcement with strong security guarantees can go hand-in-hand with provably correct, classical static analysis methods, so the security-performance tradeoff can continue to be bridged.

2 The Attack

In order to motivate the encryption of the metadata table that is central to Softbound+CETS' memory safety guarantees, my project includes a simple attack embedded in an executable linked with one compilation unit unprotected by Softbound+CETS instrumentation. At a high level, given

knowledge about the location in virtual memory of the metadata table, the unprotected module can directly modify select entries of the table in order to escalate the privilege of a specific pointer. In my case, I have demonstrated this fact by modifying the table to allow, within the protected module, an offset from a stack allocated `Buffer A` to modify a disjoint section of the stack owned by `Buffer B`. While this is a simple example, I think it's clear that this is a limitation to the adoption of SoftBound+CETS instrumentation in legacy systems. Why should programmers feel justified incurring considerable overheads when any existing, unprotected module can easily faciliate privilege escalation within a module that *is* "protected"?

2.1 Relevant Background

First, in order to understand the way the attack was mounted, it is important to specify the mechamism that SoftBound+CETS uses to propagate and update metadata associated with each pointer.

2.1.1 Pointer & Metadata Origination

C semantics dictate that pointers can originate in multiple, legitimate ways. Most straightforwardly, the compiler can allocate variable, fixed-size buffers, and other statically-sized objects on the function's stack frame. Pointer variables that point to stack-allocated memory have a base, bound, and size that is determined at compile time. This metadata is inserted in-line with the pointer variable declaration.

In particular, here is my mental model of stack variable pointer transformations. With the following code:

```
struct student {
  int age;
  int id;
};

int main()
{
  struct student parth;
  parth.age = 23;
}
```

I expect to see transformed LLVM intermediate representation code to the following effect:

```
...
%1 = alloca %struct.student, align 4
%1_age = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 0

// Softbound+CETS inserted
%1_base = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 0
%1_bound = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 2
%1_size = i32 4; // each field of 'student' holds 4 bytes
...
```

Here, the `alloca` instruction is allocating stack memory for an object with size `%struct.student` and storing the pointer in `%1`. Then, the `getelementptr` (GEP) instruction is just computing a pointer offset from some base. Finally, at the point where the variable `%1_age` is finally dereferenced, a branch is also inserted to verify the condition `%1_base <= %1_age + %1_size <= %1_bound`. In this example (where `fp` stands for "frame pointer"), the check simplifies to:

```
fp + 0 <= fp + 0 + 4 <= fp + 8
```

2.1.2 Pointer Propagation

An interesting case to deal with is when programmers need to store pointer variables themselves into memory (as a concrete example, the `task_struct` in the Linux kernel holds a pointer to the `pid` struct in its field `thread_pid`).

Here, SoftBound+CETS instrumentation is required to make a *metadata table entry* for the pointer value being stored to memory. The metadata cannot be reliably inlined into the function body because the pointer value will most likely escape the scope that any inlined metadata might belong to.

In particular, the metadata table maps each pointer stored into memory to a struct with `base` and `bound` values. The mapping uses a two-level table in order to balance quick look-up and modification times with reasonable memory usage. Specifically, the container I used to mount the attack used 48 bit virtual addresses. The following diagram shows the way a pointer value is used to find its mapped metadata entry:

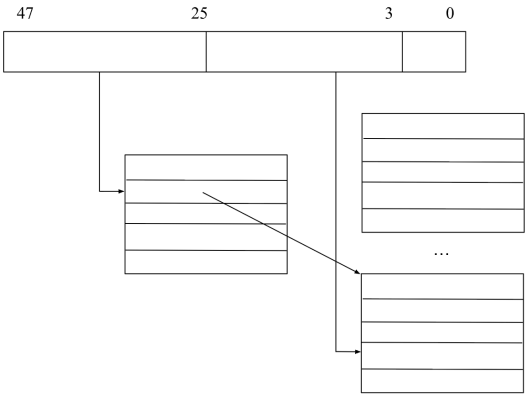

Figure 1: The Two-Level Lookup Table Structure

2.2 Attack Details

My attack employs an unprotected module to overwrite a key metadata table entry to escalate the privilege afforded to a pointer that lives in the SoftBound+CETS instrumented subset of the executable. This is not the only way to modify a pointer's metadata. For example, even the "inlined" metadata presented in 2.1.1 will most likely exist in the memory if `main` calls a subroutine (some

metadata variables might be pushed to the callee's stack frame). A malicious subroutine might, for example, write beyond its stack frame, and modify the stack location pertaining to the `%1_bound` variable in its parent's stack frame.

However, my attack exploits the metadata table because I wanted to motivate the encryption of the metadata table as an initial, low-complexity encryption target (and get a rough understanding of overhead). It is possible (and necessary, for full protection) to encrypt metadata that lives on the stack, but that was a more involved process that I discuss in Section 3.3.

In my attack, I initialize two adjacent arrays, `array1` and `array2`, and I store the pointer to `array1`'s first element is stored to memory.

```
/* main.c (protected) */

// these arrays live next to each other on the stack
int array1[] = {1, 2, 3, 4};
int array2[] = {5, 6, 7, 8};

// ensure pointer 'array1' has an entry in the metadata table by
// storing the pointer to a location in memory (i.e. not just inlined)
int *array1_ptr[] = {array1};
```

Next, I call the malicious subroutine in my unprotected module. Within this subroutine, having turned off ASLR, I hard-coded the address of the first-level metadata table. I then descended the two-level table, found the entry, and widened my privilege by 4 bytes in both directions.

In my protected module, I then used my escalated privilege to write beyond the bounds of `array1`, and influence `array2`.

```
/* main.c (protected) */

// call malicious subroutine
overwrite_metadata_map(array1_ptr, 4);

int *array1_alias = array1_ptr[0];
*(array1_alias - 1) = RANDOM_INTEGER;
*(array1_alias + 4) = RANDOM_INTEGER;
```

We can see the change from the stores above in the image below (`RANDOM_INTEGER = 37`).

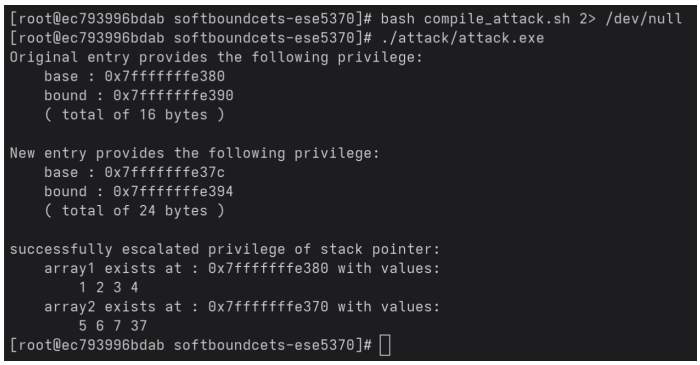

Figure 2: Successful Attack: Escalate Afforded Privilege & Overwrite Neighboring Buffer

The directory with the code can be accessed here:
`https://github.com/parthsarkar17/softboundcets-ese5370/tree/main/attack`.

3 Encryption

As previously mentioned, my goal with encrypting the table was to prevent illegal modification of any table-stored metadata entries, which was the mechanism I used to mount the attack detailed above. Importantly, encryption does not prevent denial of service; an attacker might overwrite legitimate, encrypted data with nonsense bytes that halt the program when later decrypted and used. However, the main goal was to prevent the targeted escalation of privilege I demonstrated above.

Based on the two-level metadata table structure introduced in 2.1.2, there are several candidates for encryption. We can encrypt the contents of the first-level table (i.e. pointers to the second-level tables) or the contents of the second-level tables (i.e. the metadata themselves). In this project, I elect to only encrypt the first-level table. I thought it was most important for a malicious program to not be able to access the metadata entry *at all*, compared to being able to access it and not being able to use or modify it.

Next, I will introduce and evaluate my two encryption methods on the metadata table.

3.1 Exclusive OR (XOR) Encrypted Table

The XOR (`^`) operator can be used as a basic encryption method. For a bit `b` and a one-bit key `k`, the operator has the property that `b ^ k ^ k = b`. Applied bitwise to a 64-bit value, a 64-bit key can provide cheap (i.e. one cycle ALU operation), symmetric encryption on sensitive data.

My XOR encryption scheme works as follows. First, I hard-coded a key `K` in the Softbound+CETS runtime library. On initialization of the first-level metadata table, I XOR each `NULL` pointer entry with `K`. Then, on every subsequent write and read from the table, I invoke functions that simply XOR the data with `K`. Since these functions only perform the XOR, I prompted the compiler to inline these calls. The key and functions I inserted are available here.

While I simply hard-coded `K` to some raw 64-bit value here, it is obviously important to generate a random value on each process initialization (in fact, Linux has a system call to generate a secure random number). Broadly, both my encryption efforts were focused on benchmarking performance, and the hard-coded value suffices for that purpose.

3.2 AES Encrypted Table

As a more serious form of protection, I then used AES to encrypt the pointers in the first-level metadata table. At a high level, AES is a symmetric encryption algorithm that encrypts a 16 byte block in 10 rounds, with a 128-bit key. The encryption procedure begins by expanding the key into 11 distinct 128-bit keys, one for each round and one additional. Then, each of the rounds applies an obfuscating function to the *state matrix* that initially encodes the plaintext. The resulting ciphertext is a highly secure, and it serves as the gold standard for my effort to encrypt the metadata table.

Because AES was more strict with its data format than the simple XOR above, I had to modify the first-level metadata table to store 16-byte blocks, instead of the 8-byte pointers it stored before. As such, I changed the first-level table from holding pointers to holding a fat pointer struct, with a size of 16-bytes.

Given more time, I would have liked to actually implement the AES encryption algorithm myself. Instead, I found a commonly-used, somewhat optimized C++ AES repository, and I modified it to link correctly with the Softbound+CETS C runtime library. My modifications to the runtime using the AES library are available here.

3.3 Results & Evaluation

I evaluate my implementation with respect to three metrics: memory overhead, concrete performance overhead, and security.

3.3.1 Memory Overhead

As an initialization step, Softbound+CETS sets up the first-level metadata table with `NULL` entries. As shown in Figure 1, the first-level table must be prepared to map bits `[47:25]` of a 48-bit pointer, for a total of 23 bits. Since each entry holds either an 8-byte `NULL` value or an 8-byte pointer to the corresponding second-level table, this first-level table occupies a total of $2^{23} \cdot 8 = 67$ MB by default. This overhead does not change for the XOR implementation. However, as mentioned in section 3.2, I needed to expand the each pointer entry in the first-level table to include 8-byte padding, in order to accomodate AES's 16-byte plaintext requirement. Therefore, my AES encryption method begins *every* process with a total of $2 \cdot 2^{23} \cdot 8 = 134$ MB of overhead by default.

3.3.2 Performance Overhead

While the overhead required by AES encryption over standard Softbound+CETS is large, I was expecting an even larger overhead when it came to performance.

The first benchmark I used was the LINPACK linear system solver. This benchmark computes the LU factorization of a user-defined matrix A using Gaussian elimination, and then solves the linear system $Ax = b$ using this factorization. While this benchmark is not the diverse and varied set a real benchmark suite should aim to be, I did indeed discover some AES performance overhead

for this benchmark. The following graph plots the raw time taken to factor and solve with an $n = 1000$ square matrix, for baseline `clang` (i.e. no Softbound+CETS), followed by my XOR and AES encrypted versions of Softbound+CETS. After a warmup period of 3 runs, I captured 5 executions of 16 iterations each, and took their average and standard deviation.

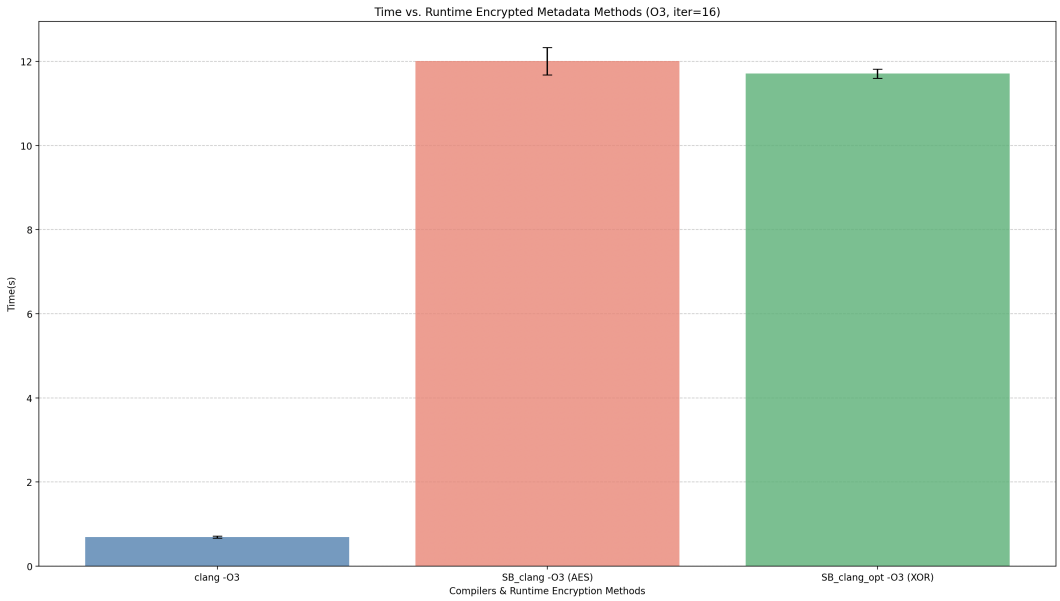

Figure 3: Time vs. Runtime Encrypted Metadata Methods (O3, iter=16)

The AES implementation took 12.006 seconds on average, the XOR implementation took an average of 11.708 seconds, and the unprotected `clang` execution took merely a 0.69 second average. For this benchmark, AES has only a 2.54% overhead relative to the XOR implementation. While this is exciting information, I have to acknowledge the following:

- First, I excluded the initial time it took to encrypt the first-level metadata table. Since this was an upfront, non-recurring cost, I considered the overhead to be sufficiently amortized over the course of the program.
- Second, this small overhead might be a limitation of the benchmark itself. It is possible that the number of pointers stored to memory in LINPACK is too small, and not representative of the "average" workload. Given more time, I would have liked to retry this experiment with a more representative suite.

Worst Case Program: To make some headway on the second bullet point, I decided to create a synethtic "worst-case" program: a linked list traversal. Since every store of a pointer to memory requires an encrypted store to the metadata table, I suspected that any pointer-chasing program would incur an extreme overhead.

My program allocates a linked list with 100,000 nodes, containing numbers 0 to 99,999 respectively. It sums up these values by traversing the list, prints the sum, and then frees the nodes. The entire program has $O(n)$ complexity. The following image shows the raw time taken[1] with `-O3`:

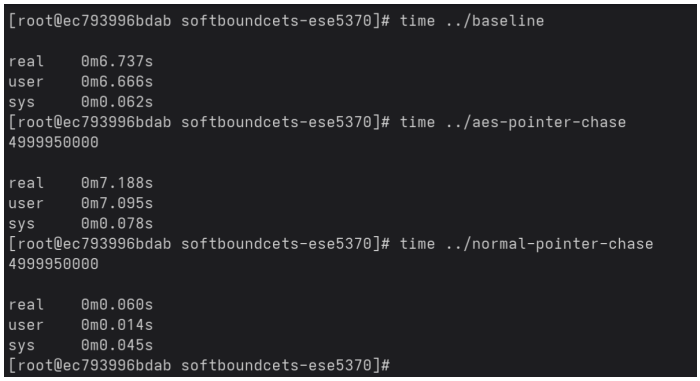

Figure 4: Pointer Chase Performance (XOR vs. AES)

Here, `./baseline` is a binary that performs AES encryption but no actual computation. I use this to obtain a baseline for how long it takes to encrypt the first-level metadata table. Thus, this image shows that, for the pointer-chasing benchmark at `-O3`, we find an astronomical $\frac{7.188-6.737}{0.060} = 7.52\times$ performance degradation.

3.3.3 Security Guarantees

The XOR and AES implementations both succeed in thwarting the attack I introduced in Section 2. My attempt to access the second-level table by indexing into the first-level table was stopped by each encryption mechanism, and resulted in a SegFault.

However, while the XOR implementation succeeded in halting the attack, has a much smaller memory footprint, and features encryption that requires only a one-cycle operation, it is also true that it offers quite weak encryption. For example, all an attacker must do to uncover the key is determine a mapping in the first-level table that *must* be `NULL (= 64'b0)`. Then, after observing the corresponding encrypted first-level table entry, it is trivial to deduce the key.

AES, however, is much more secure. As we have seen in class, its main attack vectors are side-channel (e.g. power) leakages to reveal keys at specific rounds. If given more time, I would have liked to extend AES encryption to cover stack-allocated metadata, thus providing complete metadata encryption for Softbound+CETS.

3.3.4 A Word on Stack Metadata Encryption

Initially, my goal for this project was to have end-to-end encryption of all metadata values in memory, making it impossible for a malicious unprotected module to mount a targeted escalation of privilege. Then, because every dereference of a pointer to a stack-allocated piece of memory might need to

[1] The image shows me running "normal-pointer-chase", but this was actually a binary with XOR encryption enabled. I didn't think it was necessary to distinguish between the normal Softbound+CETS and XOR versions because the overhead would be negligible for standard XOR operations (that are inlined, so no stack frame overhead either).

decrypt metadata that exists on the stack, I wanted to use the high overhead of encryption and decryption to motivate an LLVM pass that cleverly elides certain dereferences to the stack memory (next section).

Unfortunately, stack protection was tricky to nail down. Since the basic LLVM IR is machine agnostic, it is unclear whether a variable will be pushed to the stack or not; hence, every metadata value would need to be encrypted before being assigned to a variable. It was ultimately too challenging to deal with all of the edge cases of variable encryption (especially considering how AES requires encryption over 128 bits, not 64) within the given time frame. Instead, I decided to move on to the compiler pass, even if I had a weaker motivation than I was hoping for!

4 Dataflow Optimization: Eliding Stack Dereference Checks

The final piece of my project was the dataflow analysis and optimization that I implemented on top of the Softbound+CETS framework. As a brief motivation, every dereference (i.e. load or store) must undergo a check at runtime to make sure the pointer, with respect to the size of the object being pointed to, lies within a base and a bound. By default, this check is implemented as a non-inlined function call; in the best case, the branch is inlined. However, it remains true that a subroutine call or branch on *every* dereference is quite expensive. Hence, it is valuable to elide dereference checks whenever possible, without giving up security guarantees. For example, as a first-order solution, Softbound+CETS already elides dereferences to global memory since pointers into that region are easy to identify, and global memory is statically sized.

Further, the idea of applying dataflow analyses is not new to Softbound+CETS; the original implementation uses LLVM's inbuilt dominator analysis to use the fact that, if a dereference of pointer p dominates a second dereference to p, then the second derefence check can be elided. My pass attempts to make a more aggressive optimization that is not provided out-of-the-box by LLVM.

At a high level, my optimization proves that type-aware dereferences of constant-offset pointers into stack memory can be elided. This is valuable for stack-allocated arrays and structs, which are relatively common progrmaming patterns. My analysis shows that checks on these dereferences can be moved from being performed at runtime, to instead being performed at compile time.

Specifically, my analysis returns the following information for each variable at each basic block (BB). If the variable is a pointer variable, either we:

1. Cannot say anything useful about the variable
2. Or, we know **(A)** the *base*, **(B)** the *bound*, and the **(C)** *size* of the object being pointed to

4.1 Intraprocedural Dataflow Theory

Understanding the theory of dataflow analyses was, to me, a critical part of my project. Specifically, this analysis did not seem like any of the ones that I was previously taught (e.g. reaching definitions, liveness analysis, constant propagation). Instead, even though it seemed straightforward, it felt like an amalgamation of interval analysis (i.e. what *range* of values can this pointer exist within?), alias analysis (i.e. which memory locations or allocation callpoints does a pointer point to?), and constant propagation. Therefore, I had some real concerns regarding the termination of the analysis and whether I would achieve any meaningful results.

The idea of flow-sensitive analyses is to determine, at every program point, what information we know about each variable, given that we could have taken any control-flow-graph traversal to get to the current point. In particular, this "information" is encoded as a map from variables in the function's scope to elements of a set that encapsulate information we want to record.

LLVM's intermediate representation is implemented in *single static assignment* (SSA) form. This means a "variable" is equivalent to (i.e. uniquely identified by) the pointer to the instruction where that variable was defined. Finally, the elements of a set to which each variable is mapped are called *abstract lattice values* (ALVs).

Further, while I actually implemented the analysis at the granularity of basic blocks as defined by LLVM, for theoretical purposes, it is easier to reason about dataflow at the granularity of each instruction. Therefore, dataflow information at "program point" i refers to the mapping from variables to ALVs just before instruction i executes.

4.1.1 Lattices & Relevance to My Optimization

For the purposes of dataflow analyses, a *lattice* refers to a set L, a partial ordering on the set $\sqsubseteq$, and a total binary operator $\sqcup$ that computes the least upper bound of two lattice elements (I will call this operator the "join" operator).

Two important elements of the lattice are "top" ($\top$) and "bottom" ($\bot$). At a given program point, if a variable is mapped to $\top$, then we cannot say anything about that variable. For example, this could mean the variable was a function input, or the variable took on different values along different routes of the CFG up to this point. On the other hand, $\bot$ means "undefined"; in particular, we can say whatever we find most convenient about that variable.

As mentioned in the introduction to Section 4, I either want to be able to map each variable to a "don't know" value, *or* to a value representing the base, bound, size, and offset into the object. First, the standard ALV $\top$ adequetely represents the "don't know" value. Then, I introduce a new ALV element that I'll call `fpoffset (ba, ptr, bo, sz)` that contains each of the above-mentioned four pieces of information.

Now, I will informally introduce my relation $\sqsubseteq$ and operator $\sqcup$. For the partial ordering relation, we intuitively want to encode which ALVs provide more information than others. As mentioned previously, $\bot$ provides the most information; we can assume anything we desire. More realistically, we then have `fpoffset (ba, ptr, bo, sz)`. If a variable is mapped to this element, we strictly know that the variable has value `ptr`, has a type `sz`, and exists within base `ba` and bound `bo`. Finally, since $\top$ gives us no information, we know $\forall x \in L, x \sqsubseteq \top$. Putting these together, we have that:

- $\bot \sqsubseteq$ `fpoffset (ba, ptr, bo, sz)`
- `fpoffset (ba, ptr, bo, sz)` $\sqsubseteq \top$

Now, given this informal ordering, the "join" operator is responsible for making sure that conflicting variable mappings do not provide too much information. So, for example, $\forall x \in L, x \sqcup \top = \top$, and $\bot \sqcup$ `fpoffset (ba, ptr, bo, sz)` = `fpoffset (ba, ptr, bo, sz)`.

4.2 Rocq Encoding & Formal Proof of Monotonicity

While the above informal definitions helped provide an intuition for my analysis throughout the implementation process, I found the formalization process immensely valuable; if I implemented my LLVM pass in accordance with my Rocq implemenation, then I would guarantee termination with useful results.

4.2.1 Monotonicity of Join

First, I encoded the lattice members precisely as described in 4.1.1:

```
Inductive alv : Type :=
    | top
    | fpoffset (base ptr bound size: nat)
    | bottom.
```

Next, the $\sqsubseteq$ relation is succinct enough to include here, and includes the intuitive definitions from above. One important addition is the `ofst_leq_ofst` rule. That proposition constructor says that, for the same pointer value, a pointer that references a larger type provides *less* information than a smaller type. The reason for this is that we want to be able to dereference a pointer as close to its bound as possible; a smaller type `size` gives us more freedom to dereference anywhere between a `base` and `bound - size`.

```
Inductive leq : alv -> alv -> Prop :=
    | bot_leq_all    :   forall a : alv,   leq bottom a
    | all_leq_top    :   forall a : alv,   leq a top
    | ofst_leq_ofst  :   forall ba p bo s1 s2 : nat,
        s1 <= s2 -> leq (fpoffset ba p bo s1) (fpoffset ba p bo s2).
```

Though the $\sqcup$ binary operation is straightforward, I will defer to the source code. Finally, given this definition of a lattice, I provided an implemenation for the following specification. It is long, so I again defer to the source file (same as above).

```
Theorem join_is_monotonic : forall x y z: alv,
    (leq x y -> leq (join x z) (join y z))
 /\ (leq y z -> leq (join x y) (join x z)).
```

4.2.2 Montonicity of Flow Rules

The next piece of the formalization involved specifying what happens to a variable's mapping when it encounters an LLVM instruction that assigns that variable. These so-called "flow rules" are also required to be monotonic. Here is my representation of LLVM instructions that assign to pointers; aside from ϕ-nodes, the first three constructors describe the complete list of instructions that compute constant offsets from stack-allocated variables:

```
Inductive llvm_instruction : Type :=
    | static_alloca (typ_size : nat) (fp_ofst : nat)
    | bitcast (typ_size : nat) (operand_alv : alv)
    | gep (typ_size : nat) (ofst : nat) (operand_alv : alv)
    | other.
```

Then, I described the flow rules in a Rocq function definition, written here. Intuitively, the rule for `static_alloca` provides a variable with a `fpoffset` ALV, `bitcast` refines a `fpoffset` value with new type information, and the `getelementptr` instruction updates a `fpoffset` with the new `ptr` value computed in the pointer arithmetic operation itself.

I provide the theorem and implementation proving the monotonicity of flow rules below. I made use of the `auto` Rocq tactic for simpler proofs that are more robust to data structure changes.

```
Theorem flow_rules_monotonic :
    forall insn, leq (flow_rule insn) (flow_rule insn).
Proof.
    destruct insn; simpl; auto;
    destruct operand_alv; simpl; auto.
Qed.
```

4.3 LLVM Implementation Details

I thought it would be valuable to briefly shed light on some specifics regarding my LLVM pass. I implemented my LLVM dataflow analysis in LLVM 12's standard C++ internal representation. Because the analysis was specific to Softbound+CETS, I included the analysis in the same file as the transformation that provides Softbound+CETS instrumentation. The analysis is approximately 450 lines of code (including abstraction behind C++ niceties, like classes). The source code is available here.

I tried very hard to abide by a direct translation of the Rocq description of my flow rules to avoid introducing any compiler bugs. The transfer function (that describes how an instruction flows variable to ALV mappings) is available here.

Next, I actually use the results of this analysis by passing an object representing analysis results to the function that performs the transformation. Specifically, I perform the analysis here, and I use the resulting object as an argument to the function `SoftBoundCETSPass::addSpatialChecks`. This transformation takes in an instruction and instruments it with a call to `spatial_dereference_check()` if it does not early-return. So, I use my analysis results precisely here, and early-return if the analysis says it's OK to do so!

4.4 Correctness, Results & Evaluation

The following sections detail the way I evaluated my optimization. In the spirit of my Rocq proofs, I felt strongly that the implementation should be thoroughly tested for correctness. I was also very curious to see what performance difference my pass would make.

4.4.1 Correctness

First, I began with hand-written tests. Since my optimization attempts to elide dereferences to constant-offset pointers to the stack, I thought of two specific ways my implementation might be broken:

I first wrote a program containing the following buffer overflow:

```
// Test Code A
int buf[] = {1, 2, 3};
buf[3] = 37;
```

As is expected, with my optimization enabled, this program fails with a Softbound+CETS-thrown exception. I then made sure my type size propagation was correctly implemented by attempting a "type confusion" style attack. Here, the pointer variable `buf` should only be able to write 3 bytes. However, I attempt to cast this pointer to an `(int *)` value, and thereby escalate the privilege by 1 byte.

```
// Test Code B
char buf[] = {1, 2, 3};
int *buf_alias = (int *)buf;
*buf_alias = 37;
```

Again, my program fails as desired.

For a more thorough and representative suite of tests, (INSERT [revisited]) provides a way to run the Juliet C Test Suite, which is a testing benchmark that contains C programs with vulnerabilities. By providing both well-formed and ill-formed inputs, compilers protecting against memory violations should succeed on valid inputs and abort on bad inputs. This subdirectory of my project contains the output of the Juliet C Test Suite's stack buffer overflow tests on standard Softbound+CETS (prefixed with `not_spa`) and Softbound+CETS with my optimization enabled (prefixed with `spa`). I performed a `diff` of the good and bad test case outputs respectively, and saw no difference from the original Softbound+CETS implementation.

Together with my Rocq implementation proving monotonicity and a sufficiently conservative analysis, I believe these two methods of testing demonstrate the correctness of my implementation.

4.4.2 Results & Evaluation

To evaluate any effects of my optimization, I used the LU-decomposition and linear solve benchmark from Section 3. For compiler flags `-O0` and `-O3`, I plotted the runtimes of the Softbound+CETS instrumented code both with and without my optimization, normalized against the execution time taken by the program compiled with my system's default `clang` binary.

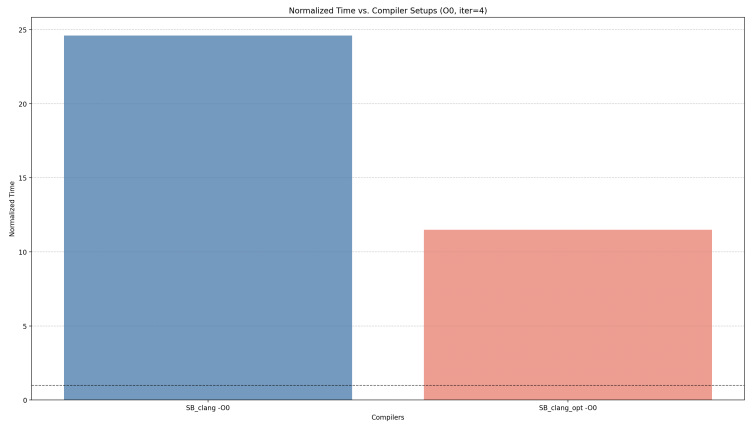

Figure 5: Normalized Time vs. Compiler Setups (O0, iter=4)

As demonstrated by the image above, my optimization provides an over 2× speedup compared to baseline Softbound+CETS in `-O0`, which was definitely a nice surprise. Another surprising result was the fact that baseline Softbound+CETS took almost 25× the normal `clang`-compiled binary's running time. This is much larger than the average of 140% cited by (CITATION; revisited).

Unfortunately, my optimization seemed to have no effect when each binary was compiled with `-O3`:

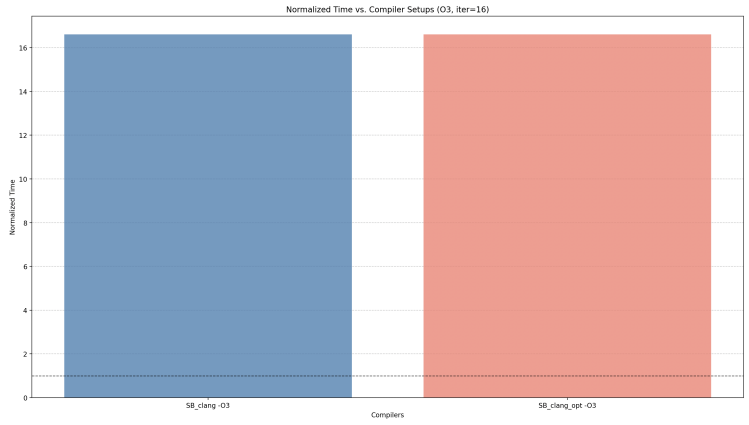

Figure 6: Normalized Time vs. Compiler Setups (O3, iter=16)

Here are my suspicions. First, `-O3` introduces a myriad of other optimizations that make it more challenging to single out the effect of any one optimization. Second, more importantly, `-O3` includes a pass that promotes many variables from living in the stack frame to living in registers. Therefore, word-sized values that we would initially have to grab from the stack frame under `-O0` (each of which previously required a Softbound+CETS dereference check that my pass had the opportunity

to optimize away) can now be retrieved *for free* because they already exist in architectural registers. These word-sized values in the stack were prime targets for my pass because pointers to these values were all constant offsets from the frame pointer. I suspect that my pass has no added effect here in `-O3` because all of these word-sized values have now been promoted into registers, and therefore no longer benefit from dereference check elision; in fact, they no longer need dereference checks at all!

Even though `-O0` typically means "no optimizations", I believe that my optimization should always be run at `-O0`, even if you disregard the 2× performance improvement. First, the reason people might want `-O0` is for debugging purposes. If that is the case, removing extraneous information like runtime dereference checks improves program readability. Similarly, people might want to avoid `-O3`, opting instead for `-O0`, because higher optimization levels usually increase code size. In my pass, code size strictly gets smaller because it never adds code; it only removes it.

5 Conclusion & General Takeaways

Broadly, I feel that my project was successful in attempting to motivate both encryption of metadata variables and the use of targeted, correct compile time optimizations to recover performance sacrificed in the pursuit of security. Given a larger timeline, I would have liked to extend my work to achieve full, end-to-end encryption of all metadata variables stored to memory. To my understanding, this would have provided the guarantee of memory safety for *all* binaries, including those only partially compiled with Softbound+CETS. While the overhead would have been very large, this further motivates the need for more clever compile time optimizations, including interprocedural optimizations (instead of intraprocedural, like the one I presented here).

Even if the overhead is too expensive to warrant deployment as real release code, in my opinion, there is great value in using Softbound+CETS as a debugging tool. Many bugs in C (even if never exploited maliciously) arise as a result of illegal memory accesses, usually leading to cryptic error messages and segmentation faults. Thus, it would be nice to have an active community around Softbound+CETS that routinely upgrades the infrastructure to more recent LLVM versions and plugs in nicely to `clang`.

Personal Fulfillment

I deeply enjoyed the process of implementing my dataflow pass, researching the relevant theory, and formally proving important properties about it. The process helped me feel more confident in my implementation as I was writing it, and it generally strengthened my knowledge and interest in the subject matter. Additionallly, it was rewarding to learn about and work with the LLVM internal representation. It was not as daunting as I expected, and I now feel more confident approaching tasks that might be better suited as an LLVM-level analysis.

Bibliography

"LLVM Link Time Optimization: Design and Implementation — LLVM 20.0.0git Documentation." *Llvm.org*, 2024, llvm.org/docs/LinkTimeOptimization.html.

"LLVM: Topics." *Llvm.org*, 2026, llvm.org/doxygen/topics.html. Accessed 30 Apr. 2026.

Lospinoso, Josh. *C++ Crash Course : A Fast-Paced Introduction*. San Francisco, Ca, No Starch Press, Inc, 2019.

Møller, Anders, and Michael Schwartzbach. *Static Program Analysis*. 2022.

Nagarakatte, Santosh, et al. *CETS: Compiler-Enforced Temporal Safety for C*.

---. *SoftBound: Highly Compatible and Complete Spatial Memory Safety for C*. 2009.

Orthen, Benjamin, et al. "SoftBound+CETS Revisited." *Fraunhofer-Publica (Fraunhofer-Gesellschaft)*, 4 Apr. 2024, pp. 22–28, https://doi.org/10.1145/3642974.3652285. Accessed 30 Apr. 2026.

Sampson, Adrian. "CS 6120." *Cornell.edu*, 16 Dec. 2025, www.cs.cornell.edu/courses/cs6120/2025fa/. Accessed 30 Apr. 2026.

---. "LLVM for Grad Students." *Cornell.edu*, 2015, www.cs.cornell.edu/~asampson/blog/llvm.html.